



Chapitre 8

Enum, struct, classes et surcharge des opérateurs

Structure du chapitre



- 1. struct - regrouper des données
- 2. `std::pair<T1, T2>`
- 3. enum class
- 4. Fonctions membres - rattacher le comportement aux données
- 5. public vs private - protéger les invariants : de struct à class
- 6. Constructeurs et destructeurs - un objet valide dès sa création
- 7. Surcharge des opérateurs
- 8. Compilation séparée - .h / .cpp

```
struct Section {  
    int number;  
    string name;  
};  
  
struct Chapter {  
    string title;  
    Section section1;  
    Section section2;  
    Section section3;  
};  
  
int main() {  
    Chapter chapter5 {  
        .title { "struct, enum et pair"},  
        .section1 { 1, "struct"},  
        .section2 { 2, "std::pair"},  
        .section3 { 3, "enum"}  
    };  
}
```



1. struct

struct – définition



- Le langage C/C++ permet de définir des **types structurés** grâce au mot-clé **struct**
- On définit d'abord un **modèle de structure** (type), puis on déclare autant **d'instances** (variables) que nécessaire
 - NB : souvent les instances sont aussi appelées **structures**
- Une structure regroupe un ou plusieurs éléments appelés **membres** ou **champs**, qui peuvent avoir **des types différents**
 - chaque membre a un **nom**, qui permet d'y accéder
 - contrairement aux tableaux où on utilise les index [n]

struct – définition



La forme générale d'une déclaration **struct**

- **nom_de_type** (optionnel), donne un **nom au type** structuré, ce qui permet de le réutiliser plus tard
- **declaration_membre** (optionnel), déclaration du membre, séparés par un « ; »
- **Identificateur** (optionnel), **déclare des variables** de ce type

```
struct [ nom_de_type ] {  
    [ declaration_membre; ... ]  
} [ identificateur, ... ];
```

On peut utiliser une *majuscule* initiale pour le modèle de structure.

Les membres (champs, en *minuscules*) sont stockés en mémoire dans l'ordre où ils ont été définis.

Une structure peut être vide (sans membre) mais pas très utile.

struct – exemples



- Les membres peuvent être du même type .. ou pas
- Une structure peut contenir un objet, un membre structuré ...
- ... et les membres peuvent être **const** et/ou avoir une **valeur par défaut**

[C++ Core Guideline C.12](#): Don't make data members const or references in a copyable or movable type

```
struct Coord {  
    double x, y, z;  
};
```

```
struct Date {  
    int jour;  
    int mois;  
    int annee;  
};
```

```
struct Personne {  
    string nom;  
    int age;  
    Date naissance;  
};
```

```
struct UneStruct {  
    const char cste = 'A';  
    int entier;  
    double* ptr = nullptr;  
};
```



- Comme d'habitude, une déclaration se fait sur la base du **type** et peut être **initialisée**.
- A défaut, les membres ne seront pas initialisés à moins qu'un constructeur le fasse
- Lors d'une initialisation partielle (**agrégat**), les valeurs sont utilisées par **position**. Les valeurs manquantes sont mises à « 0 »

```
Date date1; // ??, ??, ????
```

```
Date date2 = {1, 2, 2019}; // 1, 2, 2019
```

```
Date date3 = {1, 2}; // 1, 2, 0
```



```
Personne Vide; // «», ??, ??, ??, ????
```

```
Personne Anna = {"Anna", 4, date2}; // «Anna», 4, 1, 2, 2019
```

```
Personne Jean = {"Jean", 18, {11, 3, 2005}}; // «Jean», 18, 11, 3, 2005
```

```
Personne Paul = {"Paul", 27, 31, 7, 1996 }; // «Paul», 27, 31, 7, 1996
```

struct – déclaration / affectation



- L'initialisation peut également se faire par **nom** ce qui permet d'omettre certains membres

```
Date date4 = {.jour=13, .mois=1, .annee=2007}; // 13, 1, 2007
Date date5 = {.jour=13, .annee=1998}; // 13, 0, 1998
```

- L'affectation globale (tous les membres) se fait avec le symbole « = »

```
Date date1; // ??, ??, ???
Date date2 = {1, 2, 2019}; // 1, 2, 2019
date1 = date2; // 1, 2, 2019
```

struct – accès aux membres



- L'accès aux membres d'une structure se fait en combinant le nom de l'instance et le nom du membre séparés d'un « . » (en lecture comme en écriture)

```
// une date  
cout << date1.mois;  
date1.jour = 12;
```

```
// référence sur une date  
Date& refDate = date1;  
cout << refDate.jour;  
refDate.jour = 12;
```

- Lorsque qu'un pointeur contient l'adresse d'une structure, il faut d'abord déréférencer le pointeur avec l'opérateur « * » pour accéder au membre avec l'opérateur « . »
- Mais la priorité de l'opérateur « . » est supérieure à celle de l'opérateur « * »
L'opérateur « -> » fait ces deux opérations dans l'ordre et allège la syntaxe et la lecture

```
Date* ptrDate = &date1;    // pointeur sur une date  
cout << (*ptrDate).jour;    // ( ) nécessaires  
cout << ptrDate->jour;  
ptrDate->jour = 12;
```

struct – accès aux membres



Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right →
2	a++ a--	Suffix/postfix increment and decrement	
	type() type{}	Functional cast	
	a()	Function call	
	a[]	Subscript	
	. ->	Member access	
3	++a --a	Prefix increment and decrement	Right-to-left ←
	+a -a	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	C-style cast	
	*a	Indirection (dereference)	
	&a	Address-of	
	sizeof	Size-of ^[note 1]	
	co_await	await-expression (C++20)	
	new new[]	Dynamic memory allocation	
	delete delete[]	Dynamic memory deallocation	

struct – passage en paramètre, retour



- Les structures s'utilisent comme les autres variables
- Elle peut être utilisée comme **paramètre**. Mais composée de plusieurs membres, un passage par **valeur (copie)** n'est **pas le meilleur choix**.
- Pour éviter que le système ait à recopier toute la structure, un passage par **référence** sera bien plus « **léger** » (pensez au **const** 😊)
- Une fonction ne peut **retourner qu'une seule valeur** mais peut tout de même retourner des valeurs groupées dans une structure

```
bool est_bissextile (const Date& date);  
void decaller      (      Date& date, int n);  
  
Date prochaine_date (const Date& date, int n);
```

struct – comparaison



- La comparaison entre deux structures n'est pas implémentée

```
date1 < date2    // ne compile pas  
date1 == date2   // ne compile pas
```

- Il faudra prévoir une fonction de comparaison qui traitera les membres choisis

```
bool avant (const Date& lhs, const Date& rhs) {  
    if (lhs.annee != rhs.annee)  
        return lhs.annee < rhs.annee;  
    if (lhs.mois != rhs.mois)  
        return lhs.mois < rhs.mois;  
    return lhs.jour < rhs.jour;  
}
```

- Des fonctions opérateurs seront vus plus loin dans ce chapitre

```
bool operator< (...);
```

```
bool egale (const Date& lhs, const Date& rhs) {  
    return lhs.jour == rhs.jour and  
           lhs.mois == rhs.mois and  
           lhs.annee == rhs.annee;  
}
```

struct – récupération des membres



- Nous pouvons récupérer les types et les valeurs des membres d'une struct

```
struct Date {  
    int jour;  
    int mois;  
    int annee;  
};  
  
struct Personne {  
    string nom;  
    int age;  
    Date anniversaire;  
};  
  
Personne Anna = {"Anna", 4, {1, 2, 2019}};
```

```
auto [n, a, d] = Anna;
```

```
// est équivalent à  
string n = Anna.nom;  
int a = Anna.age;  
Date d = Anna.anniversaire;
```



2. `std::pair<T1, T2>`

Structure en retour de fonction



- Une fonction doit permettre de calculer le **quotient** et le **reste** de la division de deux valeurs
- La fonction ne peut pas faire **return** de plusieurs valeurs

- .. mais avec une **structure**

```
struct Quotient_Reste {  
    int quotient;  
    int reste;  
};
```

```
void divmod(int  numérateur,  
            int  dénominateur,  
            int& quotient,  
            int& reste) {  
    quotient = numérateur / dénominateur;  
    reste    = numérateur % dénominateur;  
}
```

```
Quotient_Reste divmod(int numérateur,  
                      int dénominateur) {  
  
    Quotient_Reste qr;  
  
    qr.quotient = numérateur / dénominateur;  
    qr.reste    = numérateur % dénominateur;  
  
    return qr;  
}
```

Structure en retour de fonction



- ... en initialisant la structure directement

```
Quotient_Reste divmod(int numérateur,  
                      int dénominateur) {  
  
    Quotient_Reste resultat {numérateur / dénominateur,  
                             numérateur % dénominateur};  
    return resultat;  
}
```

- ... et encore mieux, sans créer de variable intermédiaire

```
Quotient_Reste divmod(int numérateur,  
                      int dénominateur) {  
  
    return Quotient_Reste {numérateur / dénominateur,  
                           numérateur % dénominateur};  
}
```



- `std::pair<T1,T2>` permet de simplifier encore notre tâche en évitant de définir une structure `Quotient_Reste` qui ne servirait qu'ici

```
pair<int,int> divmod(int numérateur, int denominateur) {  
    return {numérateur / denominateur, numérateur % denominateur};  
}
```

- Elle est équivalente à `struct { T1 first; T2 second };`, de sorte que le code appelant l'utilise comme suit

```
int n = 42, d = 12;  
auto qr = divmod(n, d);  
cout << n << " / " << d << " = " << qr.first << endl;  
cout << n << " % " << d << " = " << qr.second << endl;  
  
auto [q, r] = divmod(n, d);  
cout << n << " / " << d << " = " << q << endl;  
cout << n << " % " << d << " = " << r << endl;
```

42	/	12	=	3
42	%	12	=	6
42	/	12	=	3
42	%	12	=	6

std::pair - définition



- `std::pair` est un **type générique** mis à disposition par la librairie `<utility>`
- Un `std::pair` contient deux membres, **first** et **second** de types quelconques

```
// Le record mondial absolu est de 56,7° C. Il a été établi à Furnace  
// Creek, dans la vallée de la mort, en Californie, le 10 juillet 1913
```

```
pair<Date, double> Furnace_Creek{{10, 7, 1913}, 56.7};
```

```
pair<Date, double> record_mondial = Furnace_Creek;  
cout << "Record de " << record_mondial.second  
    << " degrés en l'an " << record_mondial.first.annee << endl;
```

```
auto [date, temperature] = Furnace_Creek;  
auto [jour, mois, an] = date;  
cout << "Record de " << temperature  
    << " degrés en l'an " << an << endl;
```

```
Record de 56.7 degrés en l'an 1913
```



3. enum class



- Considérons le code ci-contre
- Il déborde de nombres magiques:
1, 2, 3, 4, 5, 6, 7

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: cout << "Jour ouvrable";  
            break;  
  
    case 6:  
    case 7: cout << "Week-end";  
            break;  
  
    default: cout << "N/A";  
            break;  
}
```



- Pour l'écrire plus proprement, on pourrait définir des constantes entières
- Mais C++ propose une solution plus élégante : les énumérations enum class

```
const int lundi    = 1;  
const int mardi    = 2;  
const int mercredi = 3;  
const int jeudi    = 4;  
const int vendredi = 5;  
const int samedi   = 6;  
const int dimanche = 7;
```

```
enum class Jour {  
    lundi = 1, mardi,  
    mercredi, jeudi,  
    vendredi, samedi,  
    dimanche};
```



- Pour résoudre ces problèmes, C++11 a introduit des **types énumérés fortement typés** : `enum class`

```
enum class Saison {printemps, ete, automne, hiver};  
Saison saison1;
```

- Pour utiliser les constantes d'énumération, il faut spécifier le nom de l'énumération

```
saison1 = Saison::ete; // il faut spécifier Saison::
```

- Le reste de l'affectation reste valable

```
saison1 = saison2; // si saison2 vaut ETE;  
saison1 = (Saison)1; // printemps = 0, ETE = 1, ...  
saison1 = Saison(1);
```

Note : la forme historique enum (sans class), héritée du C, existe encore dans du code existant - à reconnaître, pas à utiliser.



- Il n'y a **plus de conversion implicite** de l'énumération vers les entiers

```
int entier = saison1;
```

```
error: cannot initialize a variable of type  
'int' with an lvalue of type 'Saison'  
    int entier = saison1;  
                ^~~~~~
```

- Mais la conversion explicite est possible

```
int entier = (int)saison1;
```

- Au contraire des **enum**, avec les **enum class**, les identificateurs des constantes d'énumération peuvent être réutilisées

```
enum class DirH {gauche, centre, droite};  
enum class DirV {haut, centre, bas};  
DirH horizontal = DirH::centre;  
DirV vertical   = DirV::centre;
```

enum class – comparaison et paramètre

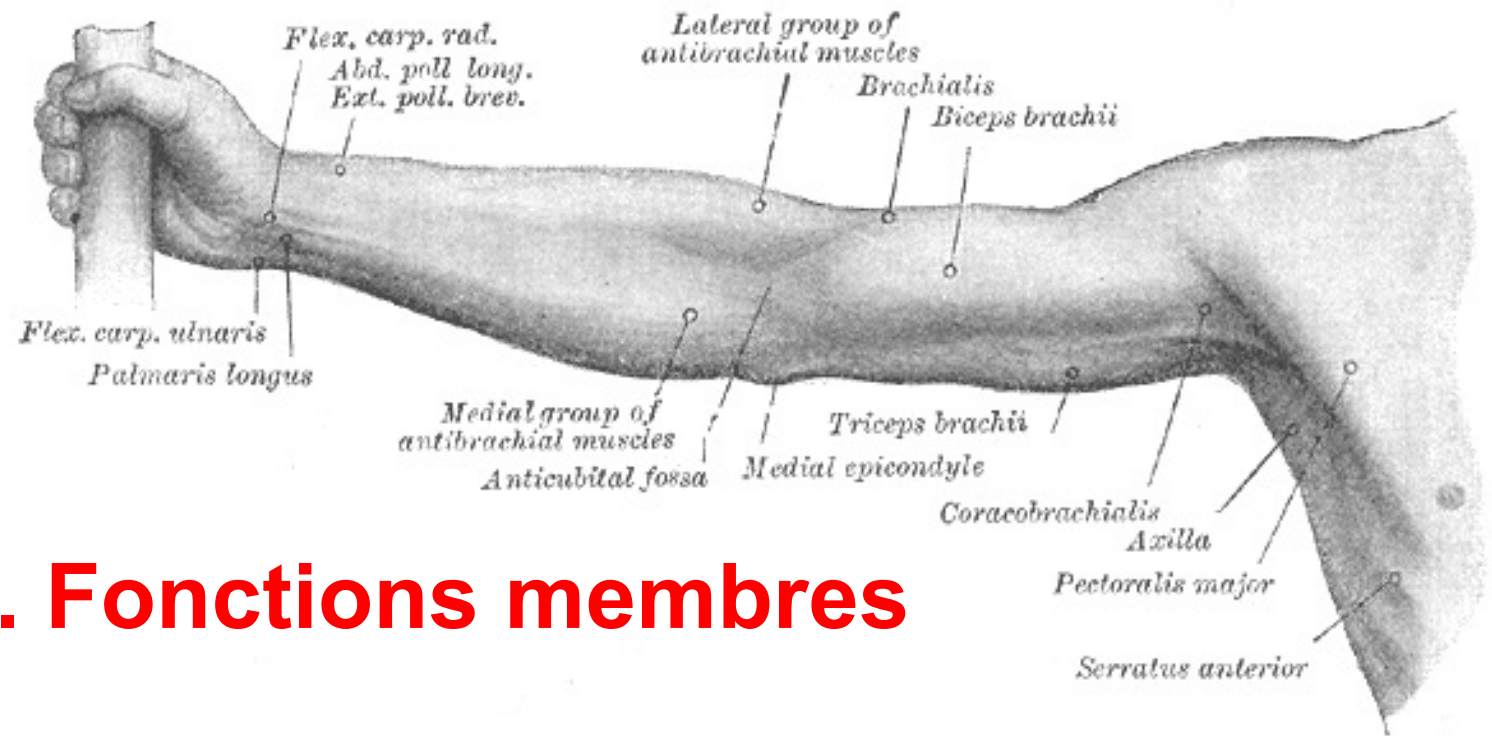


- Au final, les `enum` ou `enum class` ne sont que des entiers. Leurs comparaisons se font naturellement

```
enum class DirH {gauche, centre, droite};  
bool meme_direction (DirH lhs, DirH rhs) {  
    return lhs == rhs;  
}
```

- Le passage de paramètre se fera par copie voire par référence si besoin
- Nous pouvons retourner un `enum` ou `enum class`

```
Jour prochain_jour (Jour jour, int n = 1) {  
    return (Jour)((((int)jour + n) % 7));  
}
```



4. Fonctions membres

Enrichir les struct... jusqu'aux classes



- Dans les sections qui suivent, nous continuons à travailler avec des struct - et nous les enrichissons pas à pas :
 - Étape 1 - fonctions membres : rattacher le comportement aux données (d.afficher() plutôt que afficher(d))
 - Étape 2 - public vs private : cacher les données pour protéger les invariants (un mois reste entre 1 et 12)
 - Étape 3 - constructeurs : garantir un objet valide dès sa création
- Au bout de ce chemin, une struct dont les données sont protégées EST ce que C++ appelle une classe - struct et class ne diffèrent que par la visibilité par défaut de leurs membres



- En travaillant avec des `struct`, on crée de nombreuses fonctions prenant une `struct` en paramètre, soit par `&`, soit par `const&`

```
struct Date {  
    int jour; int mois; int annee;  
};  
  
void afficher(Date const& d);  
bool est_bissextile(Date const& d);  
Int  jours_du_mois(Date const& d);  
void incrementer(Date& d, unsigned n);  
  
int main() {  
    Date date{24, 1, 1798};  
    incrementer(date, 1905);  
    afficher(date);  
}
```

14/4/1803

```
void afficher(Date const& d) {  
    cout << d.jour << '/'  
         << d.mois << '/' << d.annee;  
}  
  
void incrementer(Date & d, unsigned n) {  
    d.jour += n;  
    while (d.jour > jours_du_mois(d)) {  
        d.jour -= jours_du_mois(d);  
        ++d.mois;  
        if(d.mois > 12) {  
            d.mois = 1;  
            ++d.annee;  
        }  
    }  
}
```



- En C++, on peut clarifier le code en transformant ces fonctions externes en fonctions membres

```
struct Date {  
    int jour; int mois; int annee;  
  
    void afficher() const;  
    bool est_bissextile() const;  
    int jours_du_mois() const;  
    void incrementer(unsigned n);  
};  
  
int main() {  
    Date date{24, 1, 1798};  
    date.incrementer(1905);  
    date.afficher();  
}
```

14/4/1803

```
void Date::afficher() const {  
    cout << jour << '/'  
        << mois << '/' << annee;  
}  
  
void Date::incrementer(unsigned n) {  
    jour += n;  
    while (jour > jours_du_mois()) {  
        jour -= jours_du_mois();  
        ++mois;  
        if(mois > 12) {  
            mois = 1;  
            ++annee;  
        }  
    }  
}
```

Comparons les 2 approches



```
struct Date {
    int jour; int mois; int annee;
};

void afficher(Date const & d);
void incrementer(Date & d, unsigned n);

void afficher(Date const & d) {
    cout << d.jour << '/'
         << d.mois << '/' << d.annee;
}

void incrementer(Date & d, unsigned n) {...}

int main() {
    Date date{24, 1, 1798};
    incrementer(date, 1905);
    afficher(date);
}
```

```
struct Date {
    int jour; int mois; int annee;
    void afficher() const;
    void incrementer(unsigned n);
};

void Date::afficher() const {
    cout << jour << '/'
         << mois << '/' << annee;
}

void Date::incrementer(unsigned n) {...}

int main() {
    Date date{24, 1, 1798};
    date.incrementer(1905);
    date.afficher();
}
```

Une fonction membre ...



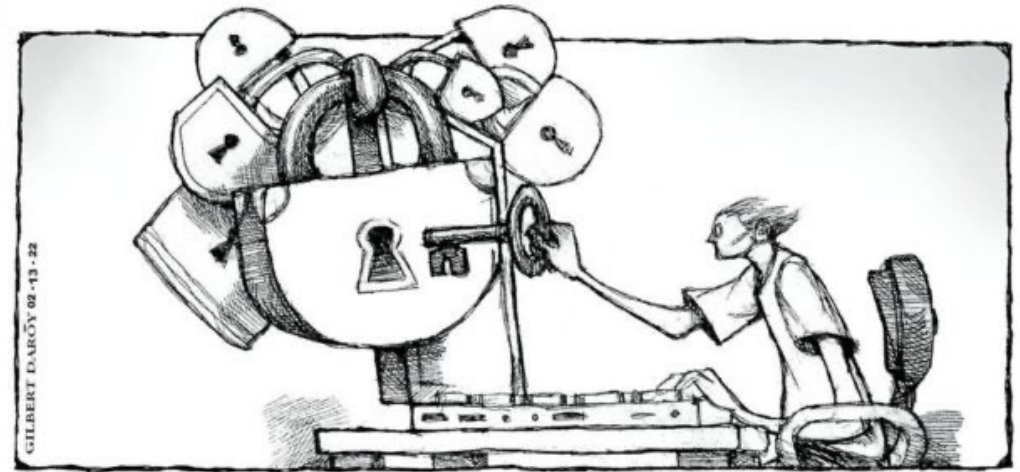
- Est déclarée dans la structure dont elle est membre
- Est déclarée `const` si elle ne modifie pas le contenu de la structure, sans qualification `const` si elle le modifie, donc pas utilisable pour des constantes
- Utilise l'opérateur de résolution de portée `::` quand on la définit en dehors de la structure. On peut aussi la définir en ligne là où elle est déclarée
- Est appelée depuis l'extérieur en utilisant l'opérateur `.`, comme pour les attributs. Ou via l'opérateur `->` si l'on a un pointeur sur la structure
- Reçoit implicitement la (variable de type) structure qui l'appelle en paramètre caché
- Peut accéder aux attributs de cette structure par leur nom, sans devoir préciser à quelle variable de type struct on se réfère
- Peut accéder aux autres fonctions membres par leur nom



- Dans le code d'une fonction membre, le mot clé **this** donne accès à l'adresse de la structure courante.
- ***this** est donc la structure courante
- Il y a donc 3 syntaxes pour accéder à un attribut ou à une fonction membre.
- Utiliser **this** est nécessaire si une variable locale cache l'attribut

```
void Date::afficher() const {  
    cout << jour << '/'  
        << (*this).mois << '/'  
        << this->annee;  
}
```

```
void Date::modifier_jour(int jour) {  
    this->jour = jour;  
}
```



5. public vs. private

Limites des structures publiques



- Une structure telle que `Coord3d` est appropriée car toutes les valeurs de `x`, `y`, `z` sont pertinentes
- Une structure telle que `Date` est problématique car `jour` et `mois` doivent respecter des règles pour faire sens
 - `mois` entre 1 et 12
 - `jour` entre 1 et `jour_du_mois()`
- La syntaxe des structures permet d'écrire `Date date{55,12,1797};`, ou de donner la valeur `13` à `mois`, ce qui n'a pas de sens

```
struct Coord3d {  
    double x, y, z;  
};
```

```
struct Date {  
    int jour, mois, annee;  
};
```

```
int main() {  
    Date date{55,12,1797};  
    date.mois = 13;  
}
```

Sections private: et public:



- Au sein de la déclaration d'une `struct` (ou d'une `class`), on peut spécifier des sections privées ou publiques avec les mots clés `private:` et `public:`
- Les attributs et fonctions membres déclarés privés sont inaccessibles depuis l'extérieur.

```
Date date;  
date.mois = 13;
```

```
error: 'mois' is a  
private member of 'Date'
```

```
class Date {  
private:  
    int jour, mois, annee;  
public:  
    void setDate(int j, int m, int a);  
    int getJour() const { return jour; }  
    int getMois() const { return mois; }  
    int getAnnee() const { return annee; }  
private:  
    bool estValide(int j, int m, int a) const;  
};  
  
void Date::setDate(int j, int m, int a) {  
    if (estValide(j, m, a)) {  
        jour = j; mois = m; annee = a;  
    }  
}  
  
bool Date::estValide(int j, int m, int a) const  
{ ... }
```

struct vs. class



- En C++, la seule différence entre une `struct` et une `class`, est qu'avant la première indication de section `private:` ou `public:`, les attributs / fonctions membres d'une `struct` sont publics, ceux d'une `class` privés.
- En pratique,
 - On utilise toujours `struct` si toutes les données sont publiques
 - On utilise toujours `class` si toutes les données sont privées
 - On évite les structures hybrides mélangeant données publiques et privées

struct vs. class



```
struct Date {  
    int jour, mois, annee; // publics  
  
public:  
    void setDate(int j, int m, int a);  
    int getJour() const { return jour; }  
    int getMois() const { return mois; }  
    int getAnnee() const { return annee; }  
private:  
    bool estValide(int j, int m, int a)  
const;  
};
```

```
class Date {  
    int jour, mois, annee; // privés  
  
public:  
    void setDate(int j, int m, int a);  
    int getJour() const { return jour; }  
    int getMois() const { return mois; }  
    int getAnnee() const { return annee; }  
private:  
    bool estValide(int j, int m, int a)  
const;  
};
```



- On appelle « objet » les variables de type « class ... »
- date1 et date2 sont deux objets de type `Date`

```
class Date {  
private:  
    int jour, mois, annee;  
public:  
    ...  
};  
  
int main() {  
    Date date1;  
    date1.setDate(24,1,1798);  
  
    Date date2;  
    ...  
}
```

Portée de la notion « privé »



- La notion de « privé » vs. « public » est définie au niveau de la `class`, pas des objets
- La fonction `Date::avant` de l'objet `*this` a accès aux données privées de l'objet `autre` parce qu'elles sont de la même classe
- Une fonction externe ou une autre classe n'y a pas accès

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    bool avant(Date const& autre) const;  
    ...  
};  
  
bool Date::avant(const Date& autre) const {  
    return annee < autre.annee ? true :  
           mois < autre.mois ? true :  
           jour < autre.jour;  
}
```

```
bool operator<(Date const& lhs, Date const& rhs) {  
    return lhs.annee < rhs.annee ? true :  
           lhs.mois < rhs.mois ? true :  
           lhs.jour < rhs.jour;  
}
```

error: 'annee' is a private
member of 'Date'



- On donne parfois accès aux attributs privés via des fonctions publiques
- Un **sélecteur** retourne la valeur d'un attribut privé. Il est typiquement déclaré **const**
- Un **modificateur** écrit dans un attribut privé. Il n'est jamais déclaré **const**
- Normalement, les modificateurs ne sont pas tous triviaux / pas tous présents, sinon il serait plus pertinent d'utiliser une **struct** avec des attributs publics.

```
class Date {  
private:  
    int jour;  
    int mois;  
    int annee;  
  
public :  
    // sélecteurs  
    int getJour() const { return jour; }  
    int getMois() const { return mois; }  
    int getAnnee() const { return annee; }  
  
    // modificateurs  
    void setJour(int j);  
    void setMois(int m);  
    void setAnnee(int a);  
    ...  
}
```



6. Constructeurs Destructeurs



- En déclarant les attributs de `Date` privés, on perd la possibilité d'initialiser l'objet `date` avec un agrégat
- Mais l'alternative

```
Date date;  
date.setDate(24, 1, 1798);
```

n'est pas acceptable, l'objet `date` étant dans un état indéterminé avant l'appel à `setDate`

- Il faut un mécanisme pour initialiser les attributs au moment de la déclaration de l'objet

```
class Date {  
private:  
    int jour;  
    int mois;  
    int annee;  
    ...  
};  
  
int main() {  
    Date date{24, 1, 1798};  
}
```

error: no matching
constructor for
initialization of 'Date'

Initialiser via un constructeur



- Un constructeur est une **fonction membre particulière** qui
 - a le **même nom** que la classe
 - ne **retourne pas** de valeur, pas même void
 - ne comporte pas d'instruction return
- Les attributs sont initialisés via la **liste d'initialisation**, précédée de :, entre la liste des paramètres et le corps de la fonction constructeur
- On peut aussi écrire dans les attributs en y affectant une valeur dans le corps du constructeur

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    Date(int j, int m, int a);  
};  
  
Date::Date(int j, int m, int a)  
    : jour(j), mois(m), annee(a)  
{  
    if (mois < 1) mois = 1;  
    else if (mois > 12) mois = 12;  
    // + autres vérifications  
}  
  
int main() {  
    Date date(24, 1, 1798);  
}
```



- Plusieurs constructeurs peuvent être définis selon les règles usuelles de surcharge des fonctions (nombres de paramètres, types, ...)

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    Date(); // valeur par défaut: Date(1, 1, 1970)  
    Date(int j, int m, int a); // jour, mois, année  
    Date(Date const& d); // copie de la date d  
    ...  
}
```

Constructeur par défaut



- Un constructeur sans paramètre est appelé **constructeur par défaut**. Il est appelé en cas

- d'initialisation par défaut

```
Date d1;    // sans parenthèses
```

- d'initialisation par valeur

```
Date d2 {}; // depuis C++11  
Date d3 = Date();
```

- Mais attention, la syntaxe ci-dessous ne déclare pas un objet d4 par valeur mais une fonction d4 retournant une **Date**

```
Date d4(); // déclare une fonction
```

- Notons qu'un constructeur peut aussi en appeler un autre via la liste d'initialisation. Une mise en œuvre alternative serait ...

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    Date();  
    ...  
}  
  
Date::Date() : jour(1),  
             mois(1), annee(1970)  
{ }  
  
int main() {  
    Date date;  
    // contient {1,1,1970}  
}
```

```
Date::Date()  
    : Date(1, 1, 1970) { }
```

Constructeur par défaut ... par défaut



- Revenons à notre code utilisant setDate plutôt qu'un constructeur ... il ne compile plus !
- La ligne `Date date`; utilisait le constructeur par défaut, qui était défini implicitement en l'absence d'autre constructeur
 - Sa version par défaut ne fait rien. Elle est équivalente à `Date::Date() {}`
- En présence d'un autre constructeur, ici `Date(int j, int m, int a)`, le constructeur par défaut n'est plus créé implicitement
- Si nécessaire, on faut l'écrire explicitement, ou utiliser sa version par défaut

```
Date::Date() = default;
```

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    Date(int j, int m, int a);  
    void setDate(int j, int m, int a);  
};  
  
int main() {  
    Date date;  
    date.setDate(24, 1, 1798);  
}
```

```
error: no matching constructor for  
initialization of 'Date'
```

Constructeur à un paramètre



- Déclarer un constructeur à un paramètre crée une **conversion implicite** du type du paramètre vers celui de la classe.
- On peut empêcher la conversion implicite en qualifiant le constructeur **explicit**. Seule une **conversion explicite** est alors possible.
- Les règles de surcharge des fonctions s'appliquent toujours quand plusieurs constructeurs sont appelables.
 - c1 utilise le 1^{er} constructeur.
 - c2 ne peut pas appeler implicitement le 2^{ème} constructeur. Il utilise le 1^{er} constructeur via une conversion implicite préalable de **int** vers **double**
 - c3 utilise le 2^{ème} constructeur car la conversion est explicite

```
class C {  
    double d;  
public:  
    C(double _d) : d(_d) {  
        cout << "d";  
    }  
    explicit C(int i) : d(i) {  
        cout << "i";  
    }  
};  
  
int main() {  
    C c1 = 2.3;    // affiche d  
    C c2 = 3;      // affiche d  
    C c3 = C(4);   // affiche i  
}
```

Constructeur de copie



- Dans le code ci-contre, quel constructeur est appelé pour initialiser date2 ?
- Le constructeur par copie prend en paramètre un objet du même type, typiquement par référence constante
- S'il n'est pas défini explicitement, le compilateur en crée une version par défaut qui copie les attributs, i.e.

```
Date(Date const& d)
: jour(d.jour), mois(d.mois), annee(d.annee)
{ }
```

```
class Date {
private:
    int jour, mois, annee;

public :
    Date(int j, int m, int a)
        : jour(j), mois(m), annee(a) {}

    Date() : Date(1,1,1970) {}
};

int main() {
    Date date1(24, 1, 1798);
    Date date2 = date1;
}
```



- Le destructeur est une fonction sans type de retour, sans paramètre, de même nom que la classe mais précédée d'un tilde : ~
- Il est appelé quand un objet est détruit, ce qui arrive quand
 - le programme s'arrête – pour une variable créée *statiquement* (i.e. *globale* ou *statique*)
 - le programme sort de la fonction où l'objet variable *locale* a été créé *automatiquement*
 - le programmeur l'efface explicitement (*delete*) pour un objet créé *dynamiquement* (*new*)
- Typiquement, on définit explicitement le destructeur pour *libérer la mémoire allouée dynamiquement* par l'objet
- Un destructeur vide `~Date() {}` est ajouté par le compilateur

7. Surcharge des opérateurs



Surcharge par fonction externe



- Comme pour les fonctions (chapitre 4), on peut surcharger un opérateur via une fonction externe
- Inconvénient:
 - l'opérateur doit être écrit en n'utilisant que l'interface public de la classe
 - sans accès aux attributs / fonctions privées

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
  
    ...  
    int getJour() const { return jour; }  
    int getMois() const { return mois; }  
    int getAnnee() const { return annee; }  
};  
  
bool operator< (Date const& lhs,  
               Date const& rhs) {  
    return lhs.getAnnee() < rhs.getAnnee() ? true :  
           lhs.getMois() < rhs.getMois() ? true :  
           lhs.getJour() < rhs.getJour();  
}
```

Surcharge par fonction membre



- Quand le premier paramètre de l'opérateur est du type de la classe, on peut l'écrire comme fonction membre de cette classe.
- Avantage : accès aux données privées
- Inconvénient : possible uniquement pour les opérateurs unaires ou binaires dont l'opérande gauche est du type de la classe. Par exemple, les opérateurs ci-dessous ne peuvent être mis en œuvre comme fonction membre:

```
class Date {  
private:  
    int jour, mois, annee;  
public :  
    ...  
    bool operator< (Date const& rhs) const;  
};  
  
bool Date::operator< (Date const& rhs) const {  
    return this->annee < rhs.annee ? true :  
           this->mois   < rhs.mois   ? true :  
           this->jour    < rhs.jour;  
}
```

```
Date operator+(int n, Date const& d);  
istream& operator>> (istream & in, Date & rhs);
```

Rappel : const



- Quand la fonction externe reçoit le 1^{er} paramètre par `const&` ou par valeur, la fonction membre est déclarée `const`.
- Quand la fonction externe reçoit le 1^{er} paramètre par `&`, la fonction membre n'est pas déclarée `const`.

```
class C {  
    ...  
public:  
    C operator- (C const& rhs) const;  
    C& operator-= (C const& rhs);  
    C& operator-- ();          // prefix  
    C operator-- (int);       // postfix  
};  
  
C operator+ (C const& lhs, C const& rhs);  
C& operator+= (C& lhs, C const& rhs);  
C& operator++ (C& c);         // prefix  
C operator++ (C& c, int);     // postfix
```

Affectation : operator=



- Doit bien se comporter en cas d'auto-affectation
- Retourne `*this`
- Est fourni par défaut pour la copie d'un objet du même type. N'est écrit explicitement que lorsque l'on gère des ressources (voir ch. 9)
- Peut affecter depuis d'autres types que celui de la classe par surcharge

```
class C {  
public:  
    C& operator= (C const& other);  
};  
  
C& C::operator=(const C& other) {  
    if (this == &other) return *this;  
    // libérer les ressources liées à *this  
    // copier le contenu de other  
    return *this;  
}
```

Accès aux éléments : operator[]



- Typiquement surchargé deux fois
 - Retourne une `const&` si l'objet est constant
 - Retourne une `&` si l'objet n'est pas constant

```
class C {  
    vector<int> v;  
public:  
    int& operator[] (size_t i);  
    int const& operator[] (size_t i) const;  
};  
  
int& C::operator[] (size_t i) {  
    return v[i];  
}  
  
const int& C::operator[] (size_t i) const {  
    return v[i];  
}
```

Surcharge opérateur <<



- Une application fréquente est la surcharge de l' **opérateur de flux <<**

```
ostream& operator<< (ostream& os, const Date& d) {  
    os << d.jour << '.' << d.mois << '.' << d.annee;  
    return os;  
}
```

```
cout << "First iPhone : " << iPhone1 << endl;  
// First iPhone : 9.1.2007
```

- Les deux paramètres sont
 - une **référence au flux de sortie** (cout, cerr) dans lequel écrire
 - une **référence constante à l'objet** à afficher
- On retourne la référence au même flux

Surcharge opérateur >>



- Ceci peut également être utilisé pour le **flux d'entrée >>**

```
istream& operator>> (istream& is, Date& d) {  
    is >> d.jour >> d.mois >> d.annee;  
    return is;  
}
```

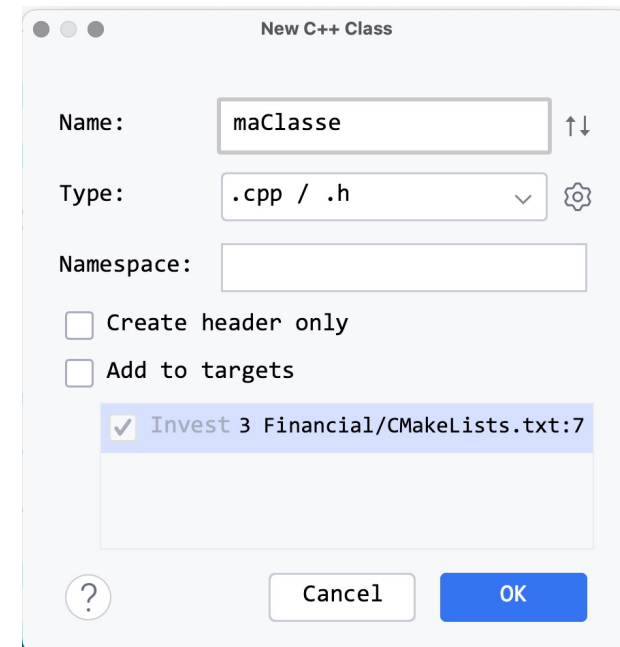
```
cout << "saisie : ";  
Date date;  
cin >> date;
```

- Le flux peut volontairement être mis en erreur si la date saisie n'est pas possible



- On n'a pas toujours le choix ...
 - `operator=`, `operator[]`, `operator()`, conversion seulement en fonction membre
 - Opérateurs dont l'opérande gauche n'est pas une classe modifiable seulement en fonction externe
- Si vous avez le choix, nous recommandons ...
 - Opérateur unaire (ex: `++`) ou binaire qui modifie son opérande gauche (ex: `+=`) :
Fonction membre
 - Opérateur binaire (ex: `+`) qui ne modifie pas son opérande gauche :

8. Compilation séparée



Organisation du code par classe



- Typiquement, chaque classe a ses propres fichiers header (.h) et .cpp, souvent nommés du nom de la classe
- La **déclaration**, dans le fichier **header**, inclut
 - les **déclarations** des données et des fonctions membres
 - la **définition** de certaines fonctions et opérateurs en ligne
- Le fichier **.cpp** inclut la **définition** des autres fonctions membres



```
#ifndef MACLASSE_H
#define MACLASSE_H

class MaClasse {
public:
    // constructeurs
    MaClasse() : constante(0) { /* ... */ }
    MaClasse(int n, double x); // defini dans maClasse.cpp
    // constructeur de copie
    MaClasse(const MaClasse& obj)
        : variable(obj.variable), constante(obj.constante) { /* ... */ }
    ...
    void fonctionMembre(int n);
    void fonctionMembreEnLigne() { /*...*/ }

private:
    int variable;
    const double constante;
};
#endif /* MACLASSE_H */
```

HE^{VD} IG maClasse.cpp



```
#include "maClasse.h"

// un des constructeurs
MaClasse::MaClasse(int n, double x) : variable(n), constante(x) { /* ... */ }

void MaClasse::fonctionMembre(int n) { /* ... */ }
```

Où placer les #include ?



- **Le fichier header** (maClasse.h)
 - inclut les headers dont il a besoin pour ses déclarations et pour les définitions *inline* des fonctions membres
- **Le fichier source cpp** (maClasse.cpp)
 - inclut le header "maClasse.h"
 - inclut les autres headers (hormis ceux déjà inclus dans le header "maClasse.h") dont il a besoin pour ses définitions des fonctions membres
- **Le fichier client** (qui utilisera la classe)
 - inclut le header "maClasse.h" et tout autre header nécessaire à son implémentation (sans prendre pour acquise l'inclusion des headers déjà inclus dans le header "maClasse.h")
- Le test `#ifndef MACLASSE_H #define MACLASSE_H ... #endif` assurera qu'on n'inclut qu'une fois chaque header, même s'il est utilisé à beaucoup d'endroits (le nom est unique)
- Et les « using namespace » ?
Il vaut mieux ne pas les mettre dans les headers, car ils s'appliqueront partout ailleurs.